



Database Management Systems

Nested Queries

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- A **nested query** can be defined as a **complete SELECT-FROM-WHERE block** within another SQL query. The other query is called the **outer query**.
- Nested queries are used to fetch existing values in the database that can be used by the outer query for comparison.
- **They can appear in the WHERE clause, FROM clause, SELECT clause, and other SQL clauses as needed.**
- SQL provides a comparison operator **IN**, which checks if a value is present in a set(or multiset) of values. **It returns True if the value is present in the set and False** otherwise. It essentially checks for Set membership.
- The **NOT IN** operator returns True if the value is not present in the set and False otherwise.

Example: Retrieve the list of project numbers of projects that have an employee with the last name 'Smith' involved either as a manager or as a worker.

Query: SELECT DISTINCT Pnumber
FROM PROJECT WHERE Pnumber IN
(SELECT Pnumber FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith')
OR Pnumber IN
(SELECT Pno FROM WORKS_ON, EMPLOYEE
WHERE Essn = Ssn AND Lname = 'Smith');

Output:

Pnumber
1
2

- Let's understand how this query works.
- Consider the first sub-query:

**(SELECT Pnumber FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith')**

- This query selects the project number of all projects handled by departments whose manager's last name is 'Smith'.
- On executing this subquery, we obtain an empty set. The output is shown below:

Pnumber (Empty Set)

- This is because there is no manager whose last name is 'Smith'.

Manager_last_name
Wong
Borg
Wallace

Consider the second subquery:

```
( SELECT Pno FROM WORKS_ON, EMPLOYEE  
  WHERE Essn = Ssn AND Lname = 'Smith' );
```

- This subquery selects the project number of all projects that employees with the last name 'Smith' work on.
- The output of this subquery is shown below:

Pno
1
2

(SELECT Pnumber FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith') → ()

(SELECT Pno FROM WORKS_ON, EMPLOYEE
WHERE Essn = Ssn AND Lname = 'Smith'); → (1, 2)

By plugging in the results of the nested queries in the outer query, we obtain the equivalent query:

SELECT DISTINCT Pnumber
FROM PROJECT WHERE Pnumber IN
()
OR Pnumber IN
(1, 2)

- In general, a nested query **returns a table(relation)** which is a set(or multiset) of tuples.
- However, there are certain scenarios wherein a nested query returns a table with **a single attribute and a single tuple**.
- In such cases, we can treat the result as a **scalar** i.e. a single value, and apply operations applicable to singular values.
- For example, the equal operator (=) is not applicable when one of the operands is a set of values(which is usually what a nested query returns).
- **However, when a nested query returns a single value, we can use the = operator instead of the IN operator.**
- SQL allows the use of tuples of values in comparisons by placing them within parentheses.

Example: Retrieve the ssn of all employees who work on any of the same (project, hours) combination that employee 'John Smith' (ssn = '123456789') works on.

Query: `SELECT DISTINCT Essn FROM WORKS_ON
WHERE (Pno, Hours) IN
(SELECT Pno, Hours FROM WORKS_ON
WHERE Essn = '123456789');`

Output:

Essn
123456789

- The IN operator compares the subtuple of values in parentheses (Pno, Hours) within each tuple in *WORKS_ON* with the set of type-compatible tuples produced by the nested query.

- In addition to the IN and NOT IN operators, a number of other comparison operators can be used to compare a single value to a set or multiset of values.
- These operators are of the form – ***comp_op* [ANY | ALL | SOME]**
- ***comp_op*** denotes comparison operators such as =, <> (not equals), >, >=, <, <= which are used to compare two values.
- ***comp_op ALL*** returns True if the result of applying ***comp_op*** between the single value and an element of the set is True **for every element of the set**. For example, the expression $1 < ALL(2,3)$ evaluates as True because 1(the single value here) is lesser than(comp_op) both 2 and 3(the elements of the set operand).
- ***comp_op SOME(or ANY)*** returns True if the result of applying ***comp_op*** between the single value and an element of the set is True **for at least one element in the set**.

Example: Retrieve the last and first names of all employees whose salary is greater than the salary of all the employees in department 5

Query: SELECT Lname, Fname FROM EMPLOYEE

WHERE Salary > ALL (SELECT Salary FROM EMPLOYEE WHERE Dno = 5);

Output:

Lname	Fname
Borg	James
Wallace	Jennifer

- First, the nested query gets executed. It returns the salary of all employees who work in department 5. The output is shown below:

Salary
30000.00
40000.00
25000.00
38000.00

- The outer query essentially becomes:

```
SELECT Lname, Fname FROM EMPLOYEE
```

```
WHERE Salary > ALL (30000.00, 40000.00, 25000.00, 38000.00)
```

- For every tuple in the *EMPLOYEE* relation, the value in the *salary* attribute is compared with the output of the nested query using > ALL. The WHERE clause evaluates to True if the value in the *salary* column for that particular tuple is greater than all the values in the nested query output. A table with the last name, first name, and salary of employees is shown to verify the result.
- The same can be achieved using the MAX function.**

Lname	Fname	Salary
Smith	John	30000.00
Wong	Franklin	40000.00
English	Joyce	25000.00
Narayan	Ramesh	38000.00
Borg	James	55000.00
Wallace	Jennifer	43000.00
Jabbar	Ahmed	25000.00
Zelaya	Alicia	25000.00

Example: Retrieve the last and first names of all employees whose salary is greater than the salary of at least one employee in department 5

Query: SELECT Lname, Fname FROM EMPLOYEE

WHERE Salary > SOME (SELECT Salary FROM EMPLOYEE WHERE Dno = 5);

Output:

Lname	Fname
Smith	John
Wong	Franklin
Narayan	Ramesh
Borg	James
Wallace	Jennifer

Notice how there are more employees who satisfy the given condition as compared to the previous example. This is because > SOME(and > ANY) evaluates as True if the employee salary in the outer query is greater than even one value in the nested query output.

- The previous example could be solved using the MIN function as well.
- = SOME or = ANY is equivalent to the IN operator.
- <> ALL is equivalent to the NOT IN operator.

- In general, we can have several levels of nested queries.
- **Ambiguity among attribute names arises if attributes belonging to the relation in the FROM clause of the outer query and the relation in the FROM clause of the nested query have the same name.**
- The rule is that a **reference to an unqualified attribute refers to the relation declared in the innermost query**

For example, consider the given query for the example :

“Retrieve the project numbers of all projects either managed by an employee named 'Smith' or worked on by an employee named 'Smith'.”

The relation PROJECT has been referenced both in the outer query and the inner query. The attributes Pnumber and Dnum in the nested query correspond to the PROJECT relation in the nested query and not the one in the outer query.

```
SELECT      DISTINCT Pnumber
FROM        PROJECT
WHERE       Pnumber IN
            ( SELECT      Pnumber
              FROM        PROJECT, DEPARTMENT, EMPLOYEE
              WHERE       Dnum = Dnumber AND
                          Mgr_ssn = Ssn AND Lname = 'Smith' )

OR

Pnumber IN
            ( SELECT      Pno
              FROM        WORKS_ON, EMPLOYEE
              WHERE       Essn = Ssn AND Lname = 'Smith' );
```

To refer to an attribute of a relation specified in the outer query, we specify and refer to an alias for that relation.

Consider the following query for the example :

“Retrieve the first and last names of employees who have a dependent with the same name and same sex as themselves.”

```
SELECT      E.Fname, E.Lname
FROM        EMPLOYEE AS E
WHERE       E.Ssn IN ( SELECT      D.Essn
                        FROM        DEPENDENT AS D
                        WHERE       E.Fname = D.Dependent_name
                                AND E.Sex = D.Sex );
```

- Here E is the alias for the relation *EMPLOYEE*. There are two attributes named Sex – one in the *EMPLOYEE* table and another in the *DEPENDENT* table.
- We must qualify E.Sex because it refers to the Sex attribute of the *EMPLOYEE* relation from the outer query, and not to the Sex attribute of the *DEPENDENT* relation.

- If there were any unqualified references to **Sex** in the nested query, they would refer to the **Sex** attribute of the *DEPENDENT* relation.
- However, we would not have to qualify attributes *such as Ssn* (of the *EMPLOYEE* relation) if they appeared in the nested query because the *DEPENDENT* relation does not have an attribute called *Ssn*, so there is no ambiguity.
- It is generally advisable to create tuple variables (aliases) for all the tables referenced in an SQL query to avoid potential errors and ambiguities

- SQL allows a subquery expression to be used in the FROM clause.
- Since a subquery returns a relation as a result, it can be inserted in a SELECT-FROM-WHERE expression at any place where a relation can appear.

Example: Find the average employee salary of those departments where the average salary is greater than \$32,000.

- One way to frame the query would be to use the GROUP BY and HAVING clauses.
- We can achieve the same by using nested subqueries. First, extract department numbers and their corresponding average salaries from the *EMPLOYEE* table using the following query:

```
SELECT Dno, AVG(Salary) FROM EMPLOYEE GROUP BY Dno
```

- The resulting relation can be then be filtered using the WHERE clause.

Query: SELECT Dno, ROUND(AVG(Salary),2) as avg_salary
FROM EMPLOYEE
GROUP BY Dno
HAVING AVG(Salary) > 32000;

Output :

Dno	avg_salary
1	55000.00
5	33250.00

NOTE: The **ROUND()** function in MySQL is used to round a number to a specified number of decimal places. Its syntax is given by ROUND(X, D) where X is the number to be rounded and D (optional) is the number of decimal places up to which X is to be rounded. If D is not provided, X is rounded to the nearest integer and returned.

Database Management Systems

Using Nested Queries



Query: SELECT Dno, avg_salary
FROM (SELECT Dno, ROUND(AVG(Salary),2) FROM EMPLOYEE GROUP BY Dno)
AS dept_avg_salary(Dno, avg_salary)
WHERE avg_salary > 32000;

Output:

Dno	avg_salary
1	55000.00
5	33250.00

SELECT Dno, ROUND(AVG(Salary),2) FROM EMPLOYEE GROUP BY Dno

- This subquery returns department numbers and corresponding average employee salaries(rounded to 2 decimal places). The output of this subquery is shown below:

Dno	avg_salary
1	55000.00
4	31000.00
5	33250.00

AS dept_avg_salary(Dno, avg_salary)

- This part of the query is used to specify the names of the result relation and its attributes. Hence, the subquery result relation is named *dept_avg_salary*, with the attributes *Dno* and *avg_salary*. Note that failing to the name this result will throw an error:

Error Code: 1248. Every derived table must have its own alias

WHERE avg_salary > 32000

- The WHERE clause selects those tuples from *dept_avg_salary* whose *avg_salary* exceed 32000.

SELECT Dno, avg_salary

- The SELECT clause selects the specified attributes *Dno*, *avg_salary* from the tuples selected by the WHERE clause.

Database Management Systems

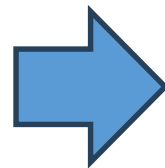
Subqueries in the From clause



- **Nested subqueries in the FROM clause are supported by most but not all SQL implementations.**
- Some SQL implementations, notably **MySQL** and **PostgreSQL**, require that each subquery relation in the FROM clause must be given a name.
- Oracle allows a subquery result relation to be given a name but does not allow renaming the attributes of the relation.
- Modifications to the written MySQL query can overcome this restriction.

MySQL

```
SELECT Dno, avg_salary  
FROM  
(SELECT Dno, ROUND(AVG (salary),2)  
FROM EMPLOYEE  
GROUP BY Dno)  
AS dept_avg_salary(Dno, avg_salary)  
WHERE avg_salary > 32000;
```



Oracle

```
SELECT Dno, avg_salary  
FROM  
(SELECT Dno, ROUND(AVG (salary),2) AS avg_salary  
FROM EMPLOYEE  
GROUP BY Dno) dept_avg_salary  
WHERE avg_salary > 32000;
```

Example: Find the maximum of total employee salaries of each department across all departments.

Query:

```
SELECT MAX(tot_salary)
FROM (SELECT Dno, SUM(Salary) FROM EMPLOYEE GROUP BY Dno)
AS dept_total(Dno, tot_salary);
```

Output:

MAX(tot_salary)
133000.00

1) What does the IN comparison operator essentially check for when used with a nested query?

- A) Arithmetic equivalence
- B) Set membership
- C) Partial string matching
- D) Null values

2) If a nested query returns a table with a single attribute and a single tuple, we can treat the result as a _____ and apply operations like the = operator instead of the IN operator.

3) In a nested query with multiple levels, if there is an ambiguity among attribute names between the outer and inner queries, what is the default SQL rule for an unqualified attribute reference?

- A) It refers to the relation declared in the outermost query.
- B) It throws an immediate syntax error.
- C) It refers to the relation declared in the innermost query.
- D) It refers to the first relation listed in the FROM clause.

4) The comparison operator combination = SOME (or = ANY) is logically equivalent to the IN operator, whereas the combination <> ALL is logically equivalent to the _____ operator.

5) When using a subquery expression in the FROM clause in MySQL or PostgreSQL, what strict requirement must be followed to avoid an error (such as Error Code: 1248)?

- A) The subquery must return only a single row.
- B) The subquery must not contain aggregate functions.
- C) Each subquery relation in the FROM clause must be given a name (an alias).
- D) The subquery cannot contain a WHERE clause.

6) The comparison operator $> ALL$ evaluates to True only if the single value is greater than every element in the set, whereas $> SOME$ evaluates to True if the value is greater than _____ element in the set.

Answers:

1. B
2. Scalar
3. C
4. NOT IN
5. C
6. at least one



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu